Programming Fundamentals

WEEK #3

Arab Academy
for Science, Technology and Maritime Transport

Lecture Agenda

- **Types of C functions**
- **Syntax of C functions**
- **Working of function**
- **Call by value**

Functions

- Functions are the building blocks of C in which all programs activity occurs.
- Function in programming is a segment that groups a number of program statements to perform specific task.
- A C program has at least one function main( ).

Types of C Functions

- Library Function
- User defined Function

Library Function

- Library functions are the in-built function in C programming system.
- For example: main(), cout<<), scanf()

User-Defined Function

- C permits programmer to define their own function according to their requirement known as user defined functions.
- A programmer wants to find factorial of a number and check whether it is prime or not in same program. Then, two separate user-defined functions in that program: one for finding factorial and other for checking whether it is prime or not.

Syntax of a Function Definition

Type-specifier function_name (parameter list)

{

Body of the function

}

Type-specifier

- Specifies the type of value that the function returns using the return statement.
- If no type is specified, the function is assumed to return an integer result.

Syntax of a Function Definition

Parameter List

- Is a comma-separated list of variables that receive the values of the arguments when the function is called.
- A function may be without parameters, in which case the parameter lists contains only the keyword **void**.

Return Statement

The return Statement

- It causes an immediate exit from the function it is in. That is, it causes program execution to return to the calling code.
- It can be used to return a value.

Syntax of return Statement

return (expression);

OR

return;

For example:

return;

return a;

return (a+b);

User- Defined Function Types

Value-Returning

- Always returns a single value to its caller and is called from within an expression.

Void

- Never returns a value to its caller, and is called as a separate statement.

Syntax of Function Call

function_name(argument(1),....argument(n));

- Control of the program cannot be transferred to user-defined function unless it is called (invoked).

Example hello.c

Function definition

Function call

```
#include <iostream>

/*
 * Print a simple greeting.
 */

void sayHello ( void )
{
  cout<<"Hello World!\n";
}

/*
 * Call a function which prints a
 * simple greeting.
 */

int main()
{
  sayHello();

  return 0;
}
```

Example: hello.c

Function name

Function body

```
#include <iostream>

/*
 * Print a simple greeting.
 */

void sayHello ( void )
{
  cout<<"Hello World!\n";
}

/*
 * Call a function which prints a
 * simple greeting.
 */

int main()
{

  sayHello();

  return 0;
}
```

Example: hello.c

Return type

Formal Parameter List

```
#include <iostream>

/*
 * Print a simple greeting.
 */

void sayHello ( void )
{
   cout<<"Hello World!\n";
}

/*
 * Call a function which prints a
 * simple greeting.
 */

int main()
{

   sayHello();

   return 0;
}
```

Example: sort.c

```
/* Print two numbers in order. */

void Sort ( int a, int b )
{
  if ( a > b )
  {
    cout<<b << a;
  }
  else
  {
    cout<<a << b;
  }
}
```

Parameters

Example: sort.c

Formal parameters

```
/* Print two numbers in order. */

void Sort ( int a, int b )
{
  if ( a > b )
  {
    cout<<b << a;
  }
  else
  {
    cout<<a << b;
  }
}
```

Actual parameters

```
int main()
{
  int x = 3, y = 5;

  Sort ( 10, 9 );
  Sort ( y, x + 4 );
  return 0;
}
```

Function Prototype (declaration)

- Every function in C programming should be declared before they are used. These type of declaration are also called function prototype.
- Function prototype gives compiler information about function name, type of arguments to be passed and return type.

Syntax of Function Prototype

return_type function_name(type(1) argument(1),....,type(n) argument(n));

int add (int a, int b);

1. name of the function is add()
2. return type of the function is int.
3. two arguments of type int are passed to function.

- Function prototype are not needed if user-definition function is written before main() function.

Working of Function

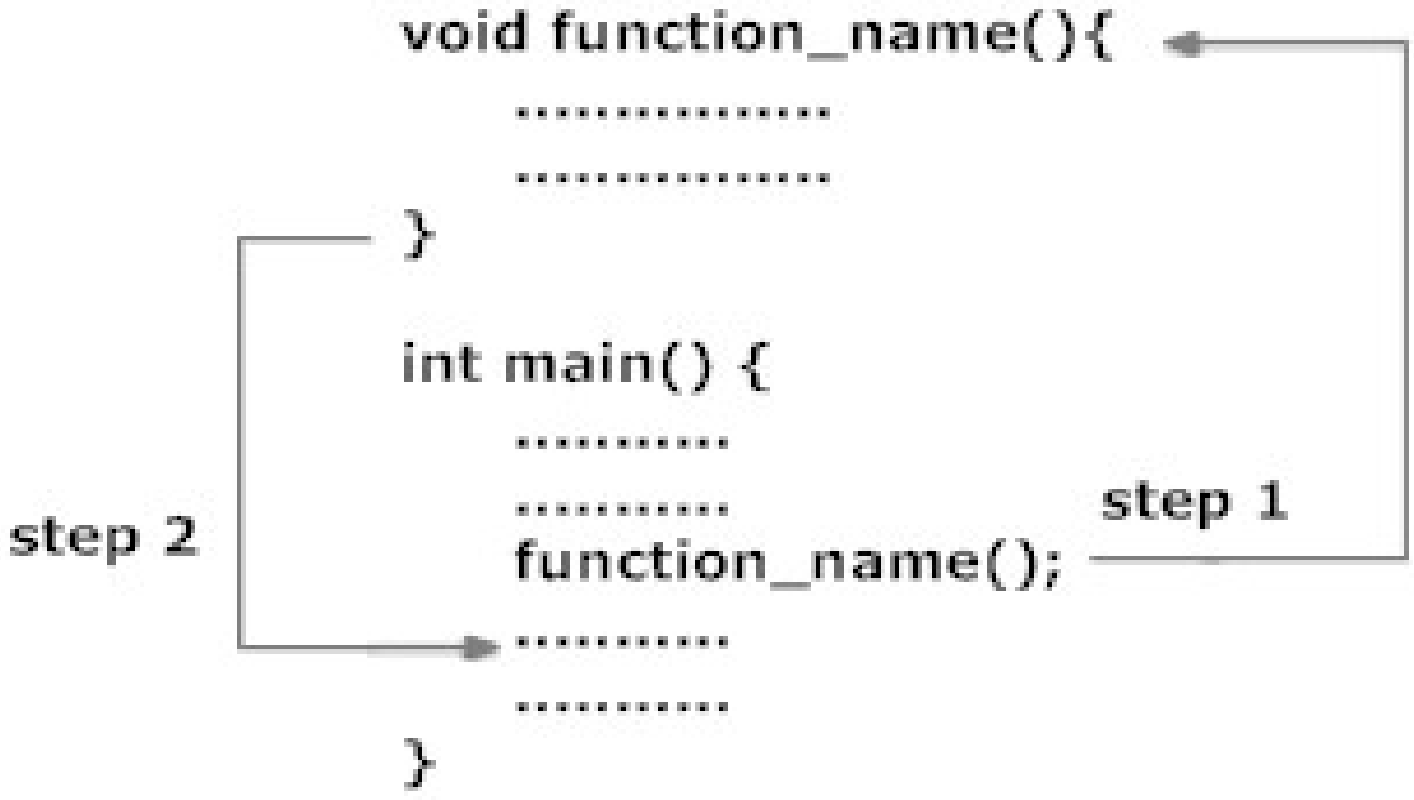

Fig: Working of Functions

Working of Function

- Every C program begins from main() and program starts executing the codes inside main() function.
- When the control of program reaches to function_name() inside main() function. The control of program jumps to void function_name() and executes the codes inside it.
- When, all the codes inside that user-defined function are executed, control of the program jumps to the statement just after function_name() from where it is called.

Call by Value & Call by Reference

Call by Value

- This method copies the value of an argument into the formal parameters.
- Changes made to the parameters of the subroutine have no effect on the variables used to call it.

Example: bad_swap.c

```
/* Swap the values of two variables.
   */

void badSwap ( int a, int b )
{
  int temp;

  temp = a;
  a = b;
  b = temp;

  cout<< a << b;
}
```

```
int main()
{
  int a = 3, b = 5;

  cout<<a << b;
  badSwap ( a, b );
  cout<< a << b;

  return 0;
}
```

Output:

```
3   5
```

Example: bad_swap.c

```
/* Swap the values of two variables.
   */

void badSwap ( int a, int b )
{
  int temp;

  temp = a;
  a = b;
  b = temp;

  cout<< a << b;
}
```

```
int main()
{
  int a = 3, b = 5;

  cout<<a << b;
  badSwap ( a, b );
  cout<<a << b;

  return 0;
}
```

Output:

```
3   5
5   3
```

Example: bad_swap.c

```
/* Swap the values of two variables.
   */

void badSwap ( int a, int b )
{
  int temp;

  temp = a;
  a = b;
  b = temp;

  cout<< a << b;
}
```

```
int main()
{
  int a = 3, b = 5;

  cout<<a << b;
  badSwap ( a, b );
  cout<< a << b;

  return 0;
}
```

Output:

```
3   5
5   3
3   5
```

Example: bad_swap.c

```
/* Swap the values of two variables. */

void badSwap ( int a, int b )
{
  int temp;

  temp = a;
  a = b;
  b = temp;

  cout<< a << b;
}
```

```
int main()
{
  int a = 3, b = 5;

  cout<<a << b;
  badSwap ( a, b );
  cout<<a << b;

  return 0;
}
```

Output:

```
3  5
5  3
3  5
```

Example: bad_swap.c

```
/* Swap the values of two variables. */

int badSwap ( int a, int b )
{
  int temp;

  temp = a;
  a = b;
  b = temp;
  cout<< a << b;
Return b;
}
```

```
int main()
{
  int a = 3, b = 5, c;

  cout<< a << b;
  a= badSwap ( b, a );
  cout<< a << b;
  return 0;
}
```

Output:

Thank You